

PARK, JOONG - HEON

PORTFOLIO

CONTACT

E-mail : oratoria989@gmail.com



정보

이름 : 박중헌

새로운 기술 탐구에 열정을 가진 AI 및 데이터 전문가 지망생으로서, 변화를 추구하며 능동적으로 행동합니다. 현재 LLM, RAG, 클라우드 인프라에 대한 깊은 관심을 가지고 전문성을 키워 나가며, 이를 활용한 창의적이고 실질적인 문제 해결에 도전하고 있습니다.

참여 프로젝트 주요 성과

- 2700여 개의 상장 기업 및 1만여 개의 스타트업 데이터 수집 및 전처리
- 9억여 건의 비트코인 트랜잭션 데이터 수집 및 전처리
- 10여 건의 프로젝트에서 클라우드 아키텍처 설계 및 구축
- 4여 건의 프로젝트에서 RAG PIPELINE 설계 및 구축
- 기존에 운영중인 기업 서비스를 개선시켜 사용자 만족도(CSAT) 50%P 향상

링크

블로그 링크:

[TECH BLOG](#)

깃허브 링크:

[GITHUB](#)

링크드인 링크:

[LINKEDIN](#)

주요 기술 스택

MAIN : 해당 기술을 사용한 3회 이상의 프로젝트 경험 보유

SUB : 해당 기술을 사용한 프로젝트 경험 보유

언어 : Python JavaScript C++

데이터 프레임워크 : LangChain LangGraph Ray Optuna Apache Spark

웹 프레임워크 : Django FastAPI Streamlit

머신러닝 라이브러리 : PyTorch OpenAI Scikit-learn Keras

시각화 라이브러리 : plotly matplotlib seaborn PyGWalker vis.js

데이터베이스 : MongoDB MySQL PostgreSQL Neo4j

Devops : LangSmith AWS Docker Git/Github Slack Jira

JOONG-HEON'S PROJECTS

[프로젝트 목차]

1. 프로젝트 소개
2. 아키텍처
3. 실험 | 흐름도
4. 성과 및 피드백

Museify

사용자 관심사 기반 문화 콘텐츠 추천 챗봇 시스템

대화형 Agent - Chatbot 웹 애플리케이션 개발

Moduler RAG 파이프라인 설계 및 구축

OCR Model Fine-tuning

01

jobAdream

기업 평가 제공 서비스

2700여 개 상장기업 및 1만여 개 스타트업의 재무/비재무 데이터 수집 및 전처리

전처리 데이터를 기반으로 성장률 예측 모델 개발

예측 및 분석 데이터 시각화

02

BTDS

비트코인 사이버 범죄 추적 수사 지원 솔루션

9억여 건의 비트코인 트랜잭션 데이터 전처리

휴리스틱 알고리즘 기반 점수 기반 결정 및 클러스터링을 이용한 거래 흐름 추적

NETWORK VIEWS 형태의 데이터 시각화

03

vAlscanbox

AI기반 멀웨어 탐지 기능 통합 스토리지 서비스

1000여 개의 멀웨어 샘플 데이터 수집 및 전처리

PE HEADER 정보와 CODE SECTION의 특징을 활용한 멀웨어 탐지 모델 개발

파일 실제 확장자 및 멀웨어 예측 결과 시각화

04

참여한 프로젝트들 중 주요 프로젝트 4개를 소개해드리고 있습니다.

MUSEIFY

사 용 자 관 심 사 기 반 문 화 콘 텐 츠 추 천 챗 봇 시 스 템

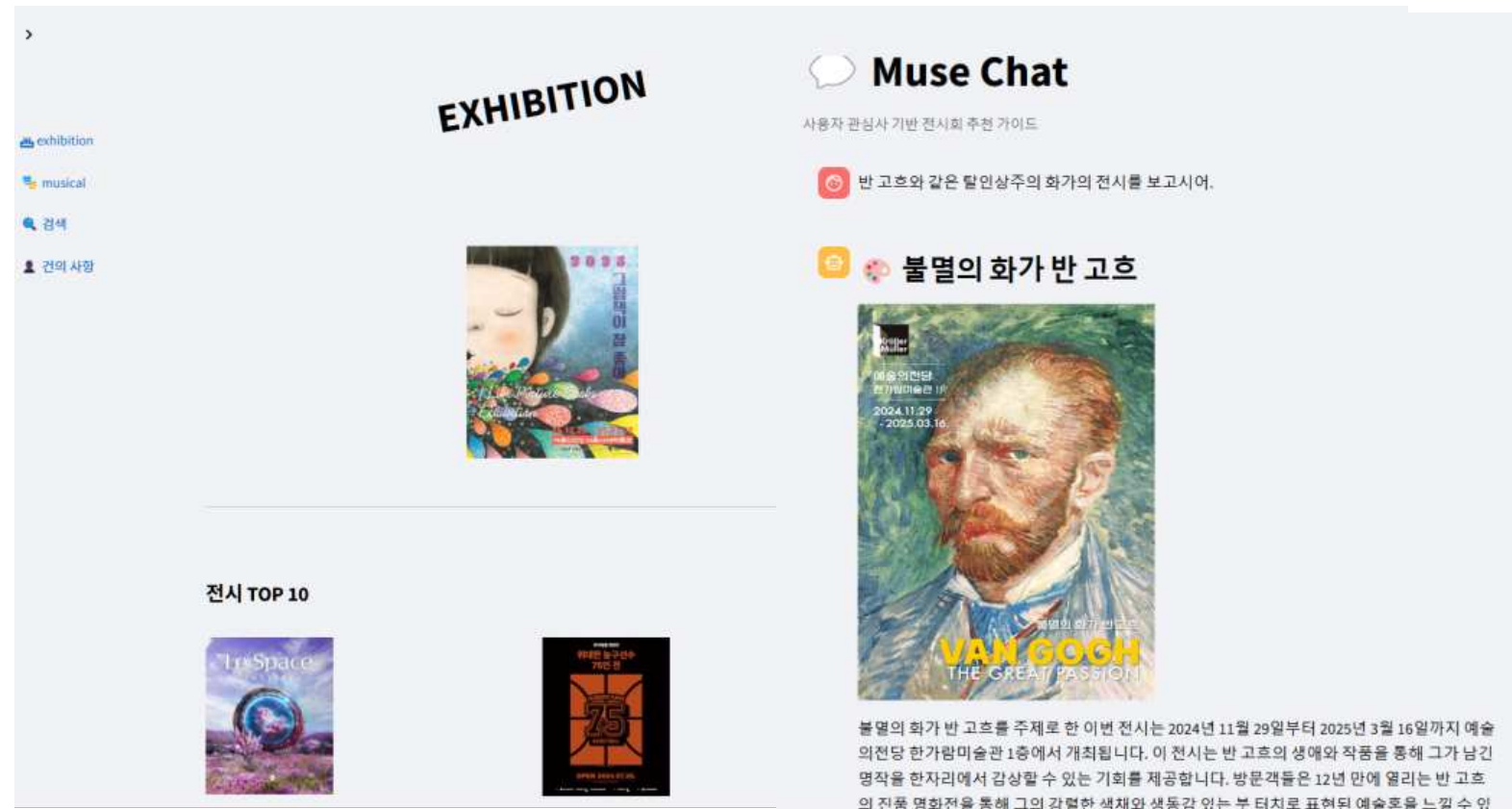
개발 인원 : 5명 (1차 AGILE) → 4명 (2차 AGILE)

기여도 : 62% (PMO, PL/RAG 파이프라인 구축, OCR 모델 파인
튜닝)

개발 기간 : 2024.11.14 ~ 2024.01.01

Museify

2030 대상 사용자 관심사 기반 대화형 문화 콘텐츠 추천 에이전트



[SITE](#) [GITHUB](#)

[기술 스택] I Used Team Used

언어 : Python

프레임워크 : LangChain LangGraph RAGAs PyTorch Streamlit

데이터베이스 : MongoDB

Devops : LangSmith AWS Docker Git/Github Zira Slack

[개요]

- 병렬 프로젝트
 - 이미지 멀티모달 LLM 전시회 추천 대화형 Agent RAG 챗봇
 - DeepFM 딥러닝 모델을 활용한 뮤지컬 추천 시스템
- 사용자 관심사 기반 추천 시스템

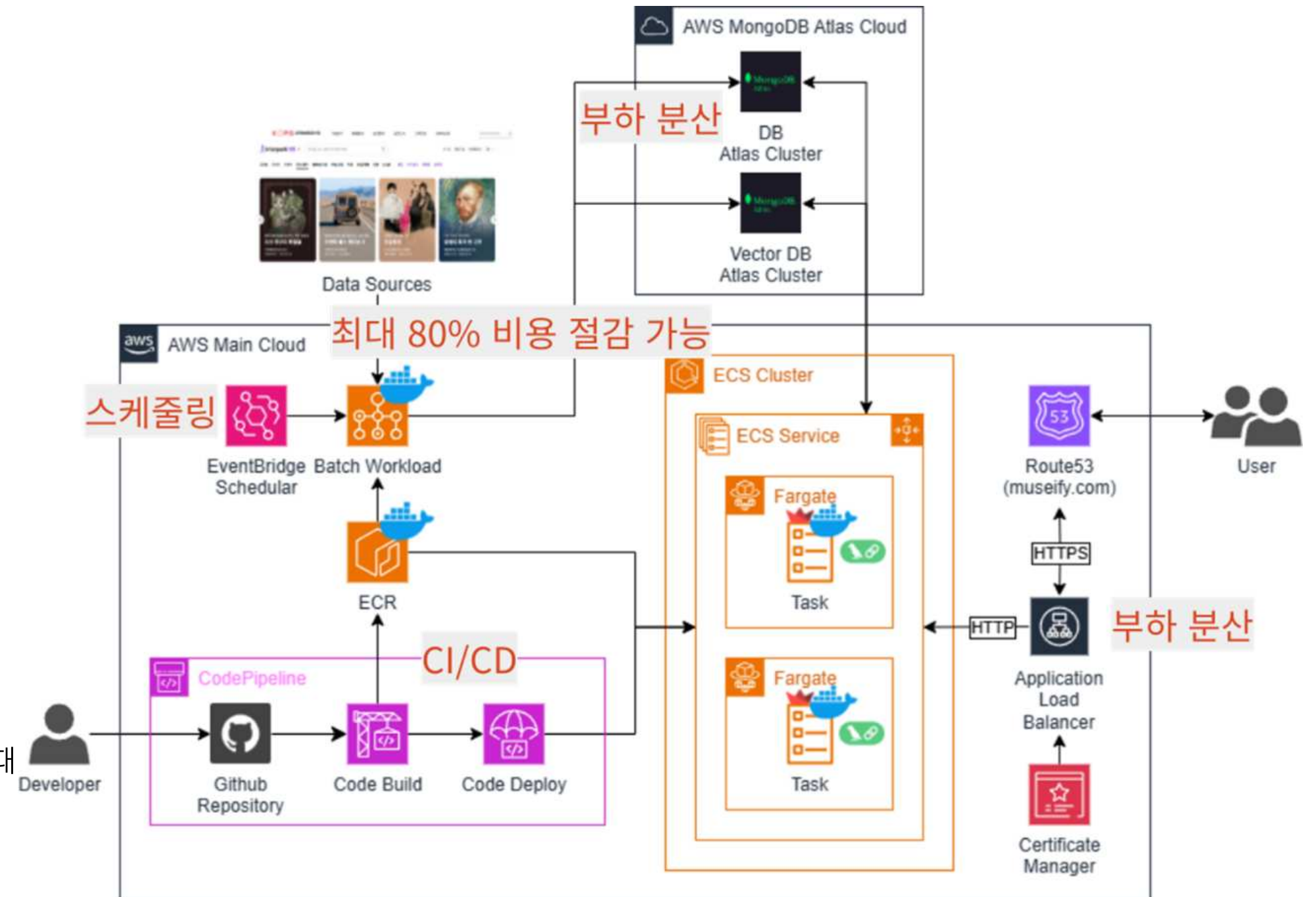
[담당 역할]

- Python을 사용한 RAG Pipeline 구축 및 OCR 모델 파인튜닝
 - CoT, GoT 기법을 응용한 Prompt Engineering
 - LangChain, LangGraph를 사용한 Modular RAG Pipeline 구축
 - PyTorch를 사용한 OCR 모델 파인튜닝
- Git/Github, Zira를 이용한 프로젝트 및 팀 관리
- AWS 클라우드 아키텍처 설계 및 구축
- LangSmith, RAGAs를 이용한 RAG Pipeline 성능 및 비용/오류 모니터링
- 이상 지표에 대한 Slack Logging Alert 기능 구현

시스템 아키텍처

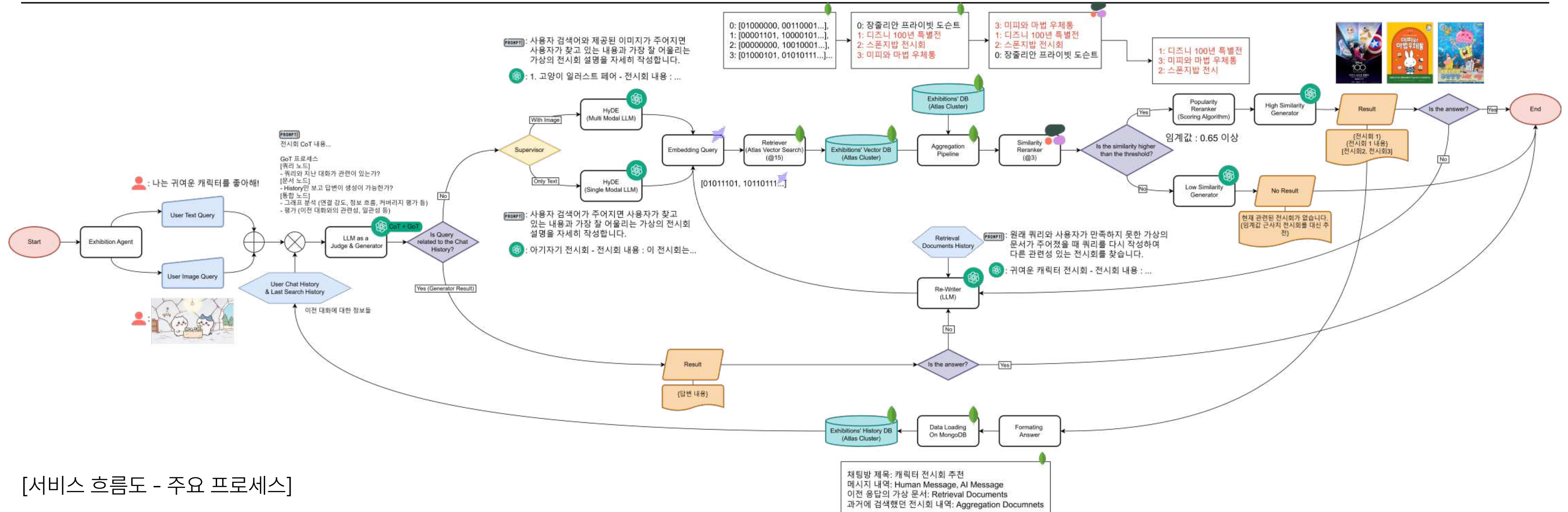
[주요 사용 서비스 및 선정 사유]

- MongoDB Atlas
 - Vector Search 기능 지원
 - 비용 효율성 (MongoDB Atlas VS MongoDB on AWS)
 - 멀티 클라우드 인프라 확장 가능
 - 유연성(유연한 데이터 구조)
 - 확장성 및 가용성 - 부하 분산(클러스터링, 샤딩)
- Batch Workload
 - 비용 절감(Fargate Spot(할인형)을 사용)
- EventBridge
 - Scheduler를 이용한 Batch Workload 자동화
- 전체 아키텍처에서 자동화, 부하 분산 및 batch 미도입 구축안과 비교했을 때
기대비용 최대 **80% 절감** 가능

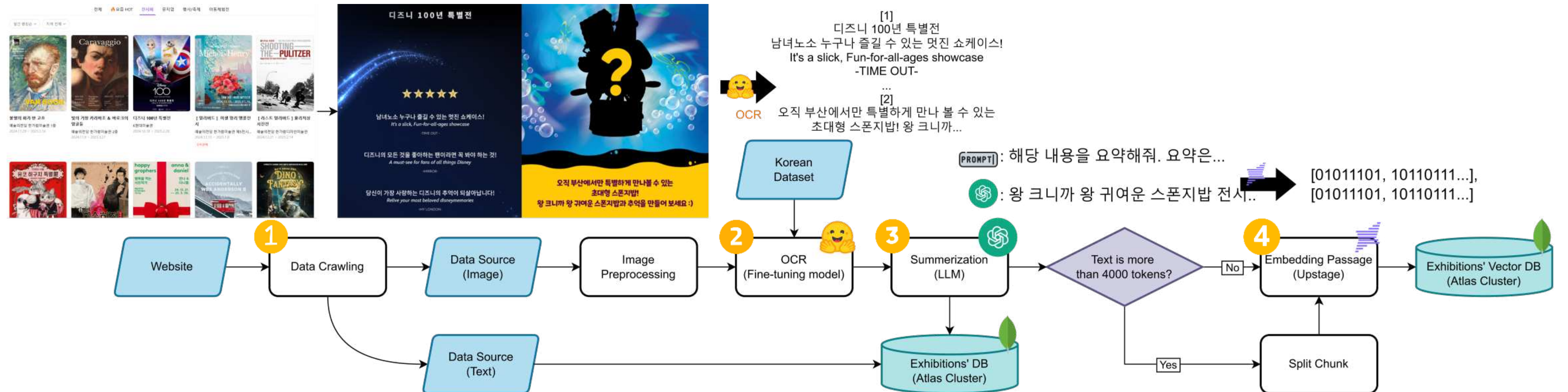


AGENT 흐름도 및 동작 예시

01

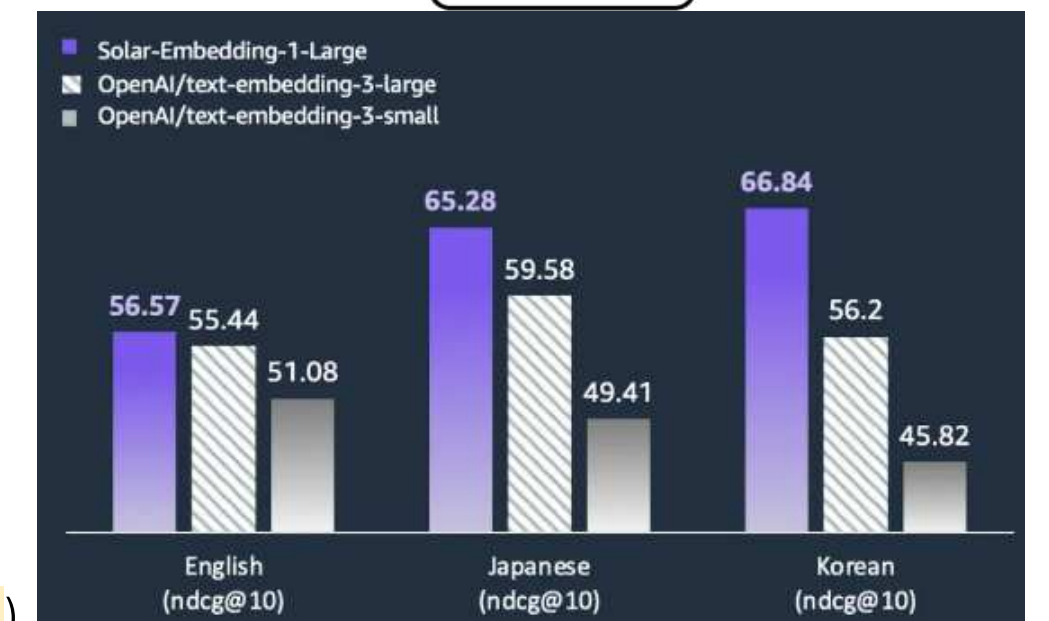


데이터 흐름도 및 동작 예시



[데이터 흐름도 - 주요 프로세스]

1. 데이터 크롤링: 웹 사이트에서 이미지(포스터) 데이터 및 텍스트(가격, 위치 등) 데이터 수집
2. OCR: 사전에 한국어 데이터셋을 가지고 Fine-tuning한 OCR model을 사용하여 이미지 텍스트 추출
 - Confidence Score를 기존 평균 0.61에서 **0.976**로 성능 개선
3. Summerization: 추출된 텍스트는 LLM을 활용하여 요약된 문장으로 기재됨,
4. Embedding: 임베딩의 경우 PoC를 진행하고 실제로 성능을 검증해보며 모델을 선정 (UpStage Korean 성능 점수 **66.84**)



<임베딩 모델 비교>

MODULAR RAG PIPELINE 구현

01

```
class Multi2HyDENode(Base):
    """가상의 문서를 생성하는 노드"""

    def __init__(self, **kwargs):
        super().__init__(**kwargs)
        self.name = "Multi2HyDENode"
        self.chain = Chain.set_hyde_chain(mode="multi")

    def process(self, state: GraphState) -> GraphState:
        hypothetical_doc = self.chain.invoke(
            {"query": state["query"], "image": state["image"]}
        )
        print("Multi2HyDENode : ", hypothetical_doc)
        return {"hypothetical_doc": hypothetical_doc}
```

```
main_graph = StateGraph(GraphState)
main_graph.add_node("single2hyde", Single2HyDENode())
main_graph.add_node("multi2hyde", Multi2HyDENode())
main_graph.add_node("sub_graph", self.sub_graph)
main_graph.add_node("judge", JudgeNode())
main_graph.add_node("supervisor", SupervisorNode())
```

<노드 할당>

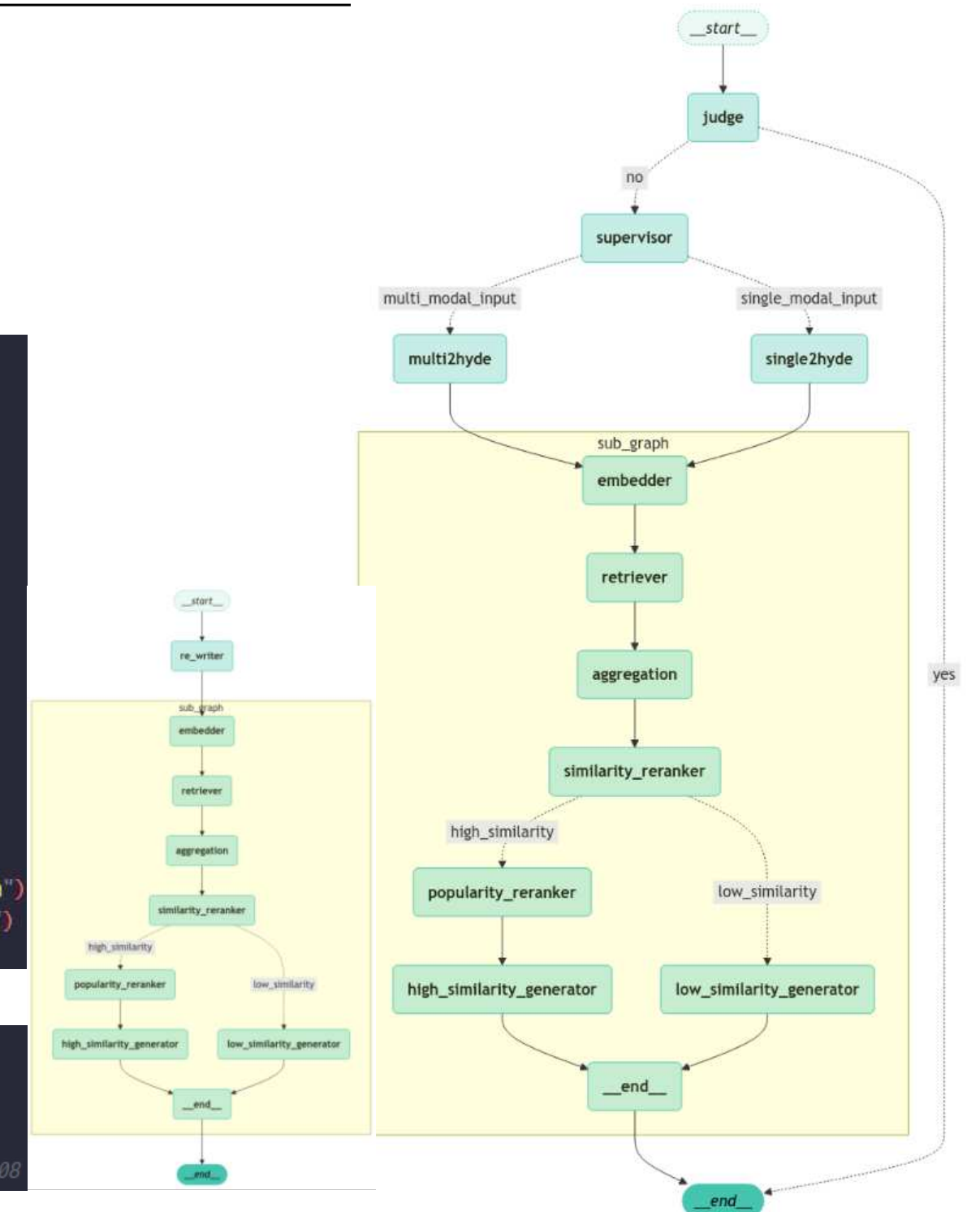
```
main_graph.add_edge(START, "judge")
main_graph.add_conditional_edges(
    "judge",
    CheckAnswer(),
    {
        "yes": END,
        "no": "supervisor",
    },
)
main_graph.add_conditional_edges(
    "supervisor",
    Supervisor(),
    {
        "single_modal_input": "single2hyde",
        "multi_modal_input": "multi2hyde",
    },
)
main_graph.add_edge("single2hyde", "sub_graph")
main_graph.add_edge("multi2hyde", "sub_graph")
main_graph.add_edge("sub_graph", END)
```

<그래프 구조>

```
metric,score,testset_size,evaluation_timestamp
context_precision,0.999999999,101,20250101_155208
context_recall,1.0,101,20250101_155208
faithfulness,0.9421645021645022,101,20250101_155208
answer_relevancy,0.7843570836008511,101,20250101_155208
```

<RAGAs Score>

*Answer Relevancy는 최대 점수 80



<RAG Flow 시각화(좌: rewrite,우:main)>

[Modular RAG] <기능별로 구분되어 있는 노드 모듈>

- 복잡한 RAG 파이프라인을 LangGraph 기반으로 노드 및 엣지 구조로 모듈화
- 사용자 쿼리에 따라 유동적으로 Multi-modal 또는 Single-modal Model을 사용하여 비용 절감 (\$1.25(multi) or \$0.075(single) / 1M input tokens)
- Retriever, Reranker 등의 노드 간 협업을 통해 효율적이고 최적화된 답변 생성.
- LangGraph로 모듈 독립적 최적화 및 효율적인 흐름 제어를 통한 속도 개선
전체 동작 속도를 1m 35s↑ 에서 30s↓로 대폭 개선
- Reranking 및 Similarity 평가를 통한 응답 품질 향상
- 사용자 기반 만족도 흐름을 반영한 'Human-Centric 추천 AI' 제안 및 수립
 - 사용자 피드백을 받아 결과를 개선하는 Human-in-the-loop 인터랙션 구현

성과 및 피드백

정량적 성과

- LangGraph를 이용하여 각 에이전트의 파이프라인 전체 동작 속도를 1m 35s↑ 에서 30s↓로 대폭 개선
- 사용자 쿼리에 따라 유동적으로 HyDE 역할을 수행하는 모델을 변경하여 비용 절감
 - 이미지가 들어올 경우 발생 비용: \$1.25(Multi)
 - 텍스트만 들어올 경우 발생 비용: \$0.075(Single)
 - 기준 : 1M Input Tokens
- RAGAs 0.2버전에서 100개의 LangChain Documents와 3개의 페르소나를 사용한 각 평가지표 부문에서 평균 점수 0.94~1.0 이상 달성
- Easy-OCR 모델을 Fine-tuning하여 Confidence Score를 기존 평균 0.61에서 0.976로 성능 개선

정성적 성과

- 기존 프로젝트에서 사용되던 일부 모듈을 재사용해 사용함으로서 Modular RAG 장점 극대화
- RAG, LLM에 사용되는 노드 및 분기 처리 함수, 도구 모듈을 저장해놓는 Module In Repository 시스템을 구축하여 다양한 파이프라인에서 사용할 수 있게끔 구현
- LangSmith 및 RAGAs를 이용하여 성능 및 비용을 Slack에서 특정 값에 대해 실시간으로 모니터링할 수 있는 자동 모니터링 Alert 시스템 개발

피드백

- [RAG Pipeline]
- LangGraph에서의 효율적인 상태 관리 방식은 더 고민해봐야 할 거 같음.
 - 다양한 페르소나를 가진 Multi Agent으로 LLM Orchestration을 통한 고도화 추천 방식 고려
 - 기억 메모리에서 Forget Ratio를 설정하여 Self-reflection이 되는Memory Retrieval을 통한 최적화 고려
- [Prompt Engineeering]
- 다양한 파생 프롬프트를 생성하는 Prompt Maker AI 와 기존에 진행했던 RAGAs 평가지표 프롬프트 개선 성과를 병합하여 프롬프트를 자동으로 개선시켜 주는 방법론 제안

JOBADREAM

기 업 평 가 제 공 서 비 스

개발 인원 : 4명 (1차 AGILE) → 8명 (2차 AGILE)

기여도 : 65% (PM/데이터 파이프라인 구축, 모델 개발)

개발 기간 : 2024.07.29 ~ (진행중)

jobAdream

구직자가 지원하려는 기업에 대한 종합적인 평가를 제공하여 정보 불균형을 해소



[GITHUB](#)

[기술 스택] I Used Team Used

언어 : Python JavaScript TypeScript Java

프레임워크 : FastAPI(Old BE) Apache Spark Vue.js Spring Boot(New BE)

데이터베이스 : PostgreSQL

Devops : AWS Docker Git/Github Swagger

[개요]

- 대기업, 중견기업, 강소기업 뿐 아니라 비재무 데이터를 활용한 중소기업 및 스타트업 성장률 예측 모델을 개발
- 이를 기반으로 구직자에게 채용 공고와 기업 평가 정보를 제공

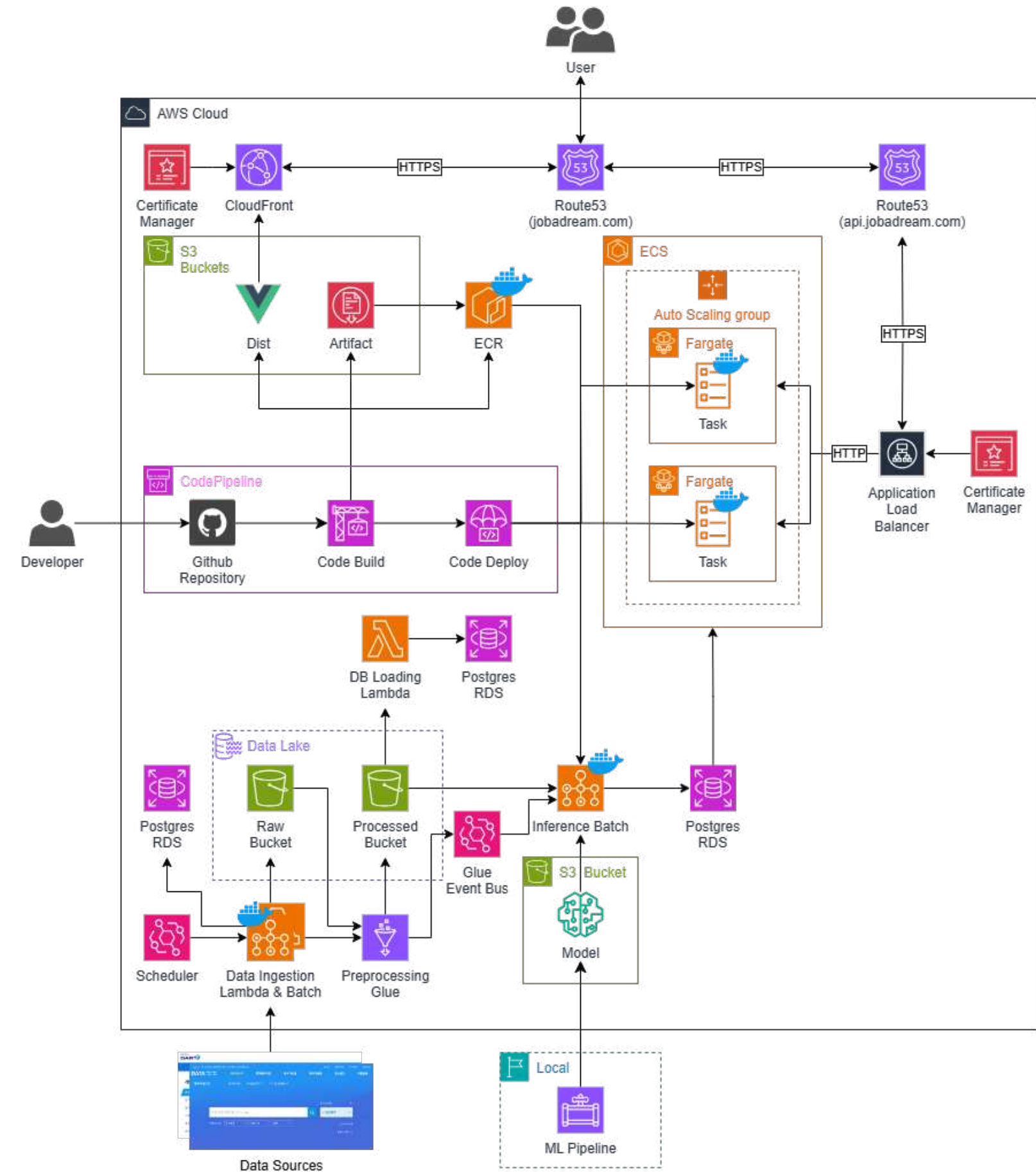
[담당 역할]

- Python을 사용한 데이터 전처리 및 모델 개발
 - AWS SDK를 이용한 데이터 수집 Lambda 모듈, Batch 워크로드 구현
 - PyGWalker를 이용한 시각적 데이터 분석
 - PySpark를 활용한 데이터 전처리 Glue 파이프라인 개발
 - Pytorch, Scikit-learn을 사용한 예측 모델 개발
- Git/Github Issue, Project를 이용한 프로젝트 및 팀 관리
- AWS 클라우드 아키텍처 설계 및 구축

시스템 아키텍처

[주요 사용 서비스]

- RDS : 실 서비스에 사용되는 데이터 보관, 관계형 데이터베이스 서비스
- S3 : 머신러닝 모델 파일, 프론트엔드 정적 파일, 데이터 레이크 파일 보관 스토리지
- CLOUDFRONT : 정적 콘텐츠와 미디어 파일의 캐싱 및 콘텐츠 배포 네트워크
- ROUTE53 : 도메인 네임 시스템(DNS) 서비스
- ECR : 백엔드 도커 이미지 저장
- ECS : FARGATE를 통해 서버리스 방식으로 컨테이너 관리 및 오토스케일링
- CODE PIPELINE : 자동화된 CI/CD 파이프라인을 구축
- ELB : 사용자 트래픽을 여러 컨테이너 인스턴스에 분산
- CM : SSL/TLS 인증서를 관리 - HTTPS 사용 목적
- LAMBDA : 데이터 수집 API FETCH 및 DB LOAD 자동화 모듈
- BATCH : 데이터 크롤링 및 모델 추론 워크로드 자동화
- GLUE : APACHE SPARK를 사용한 대규모 데이터 전처리
- EVENTBRIDGE : 데이터 수집 스케줄링 및 GLUE 이벤트 버스



데이터 아키텍처

[전체 데이터 처리 과정]

1. 데이터 수집 레이어 - Scheduler 작업 수행

수집이 종료되면 Parquet 파일 형식으로 S3에 저장

a. Batch - Crawling Workload

b. Lambda - API Fetcher Modules

2. 데이터 전처리 레이어

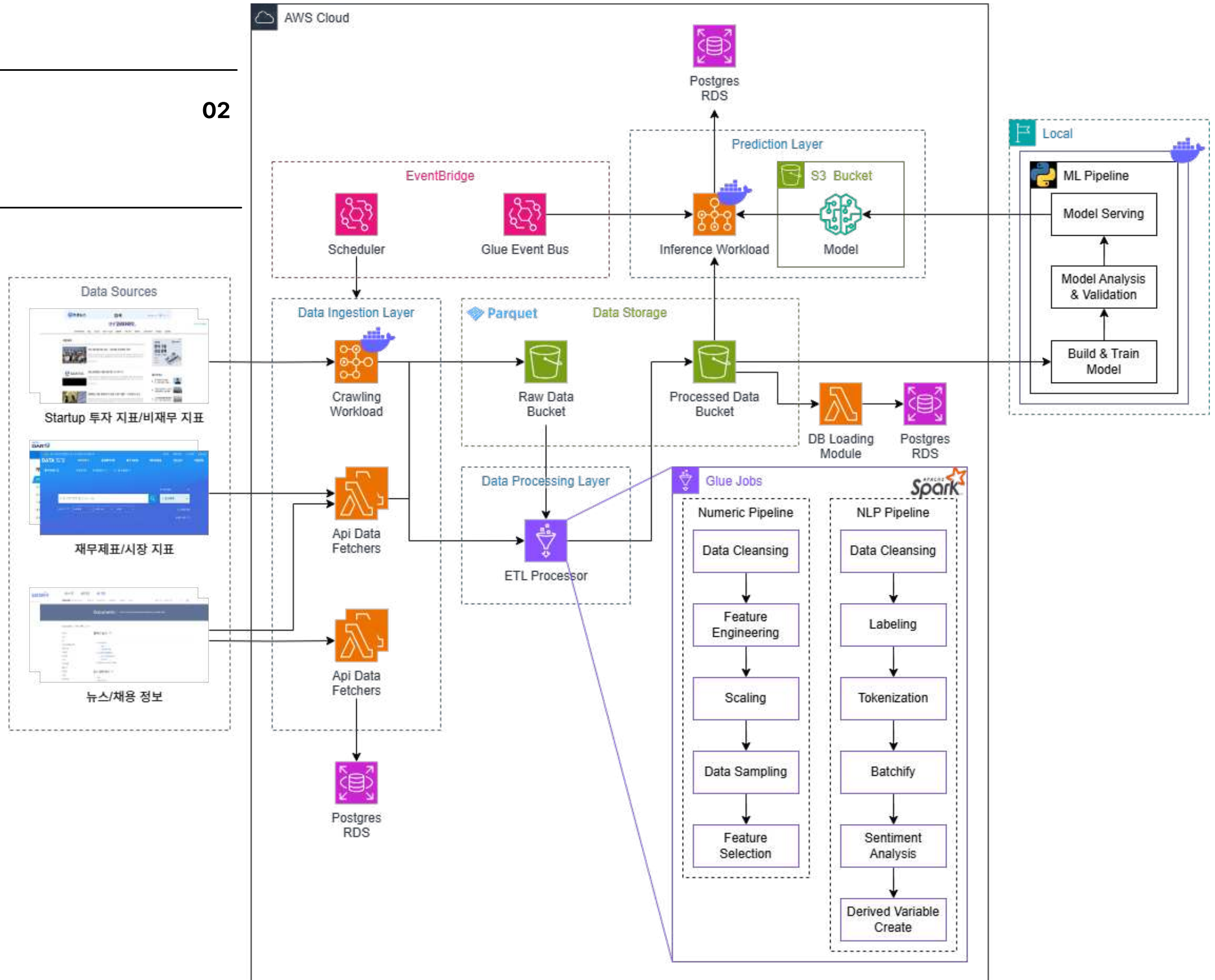
Glue를 사용하여 PySpark 기반 데이터 전처리 작업 수행

a. Numeric Pipeline - 숫자형 데이터에 대한 전처리 작업

b. NLP Pipeline - 자연어 데이터에 대한 전처리 작업

3. 인퍼런스 레이어 - Glue 작업 완료 시점에 수행

- Batch Workload에서 학습된 모델을 사용한 기업 전망 예측 결과 도출



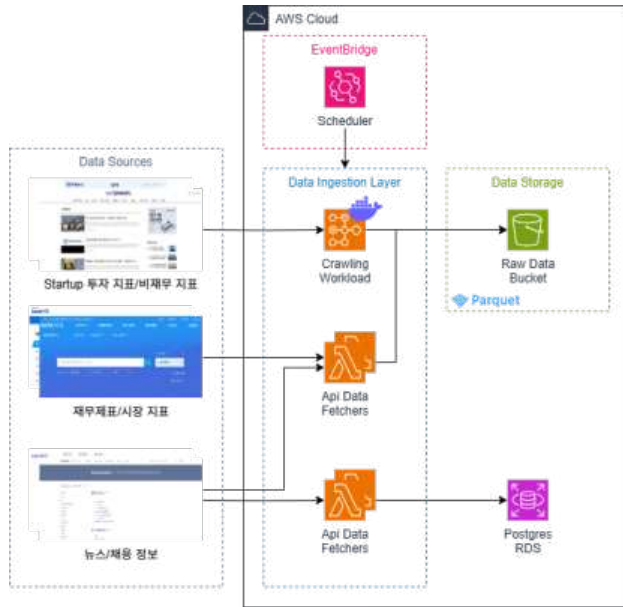
데이터 아키텍처 - 상세

[데이터 수집 - Batch, Lambda]

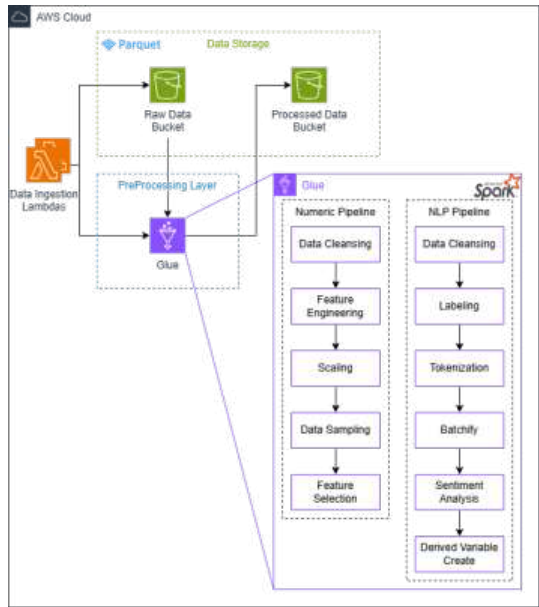
- Crawling Workload : Startup 투자 정보
- API Fetcher : 주식, 시장 지표, 뉴스, 채용 정보 등

[데이터 전처리 - Glue]

- 각 데이터는 숫자형 데이터와 자연어 데이터로 구분하여 파이프라인을 거침.
- 자연어 처리
 - 감성분석 진행을 위해 **KoFinBERT, KLUE-BERT의 성능 비교**
 - KLUE-BERT 모델로 금융 감성 LABEL을 사전학습하여 금융 감성 분석을 진행
→ 해당 분석을 토대로 문장별 LABEL 비율과 기사 수를 파생변수로 생성



<데이터 수집 레이어>



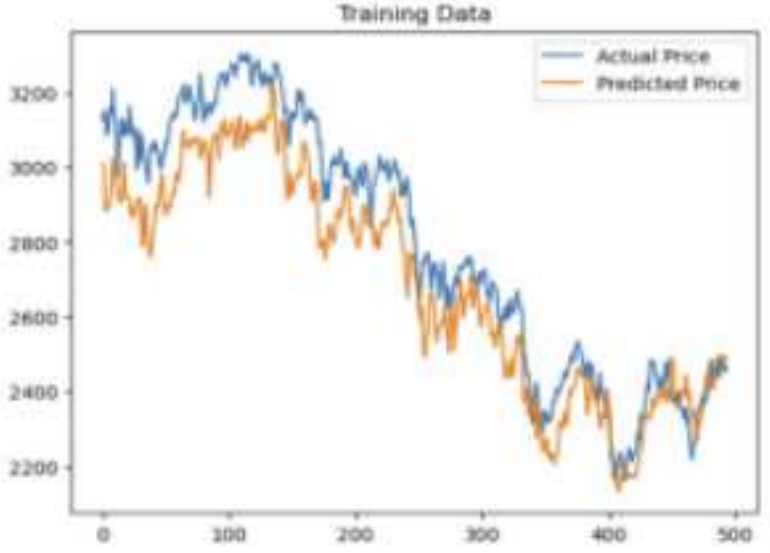
<데이터 전처리 레이어>

Sentiment Model	Text Version	Prediction Model	MSE	MAPE
KoFinBERT	title	LSTM	3345.17	1.88%
		GRU	3391.46	1.97%
		CNN-LSTM	2284.5	1.62%
		CNN-GRU	2015.82	1.17%
	summarize content	LSTM	4477.07	2.1%
		GRU	3818.08	2.07%
		CNN-LSTM	4776.8	2.29%
		CNN-GRU	2497.49	1.62%
KLUE-BERT	title	LSTM	7065.0	2.69%
		GRU	2404.66	1.6%
		CNN-LSTM	5151.5	2.48%
		CNN-GRU	2497.49	1.62%
	summarize content	LSTM	14338.72	4.03%
		GRU	4241.11	1.91%
		CNN-LSTM	3374.21	1.86%
		CNN-GRU	3374.21	1.86%

<KoFinBERT와 KLUE-BERT의 성능 비교>

기업명	기사 내 문장 예시	기업명	기사 수	긍정	부정	중립
울**설	23개사 모임으로 시작한 차세대 클럽은 10여년만인 올해 회원수가 72개로 늘었고...	울**설	40	0.82	0.10	0.07
울**설	제 월급은 대기업 다닐 때보다 반으로 줄었고요					

<뉴스 데이터 처리>



<KLUE-BERT 기반 CNN-GRU 모델 성능>

데이터 아키텍처 - 상세

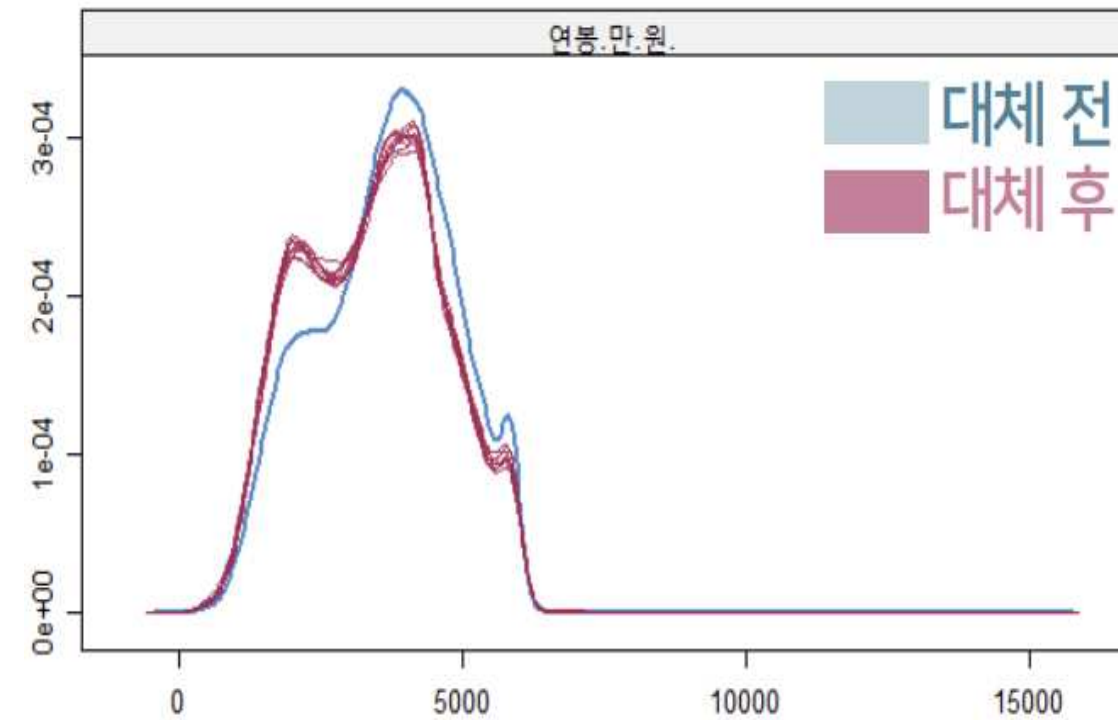
02

[데이터 분석 2]

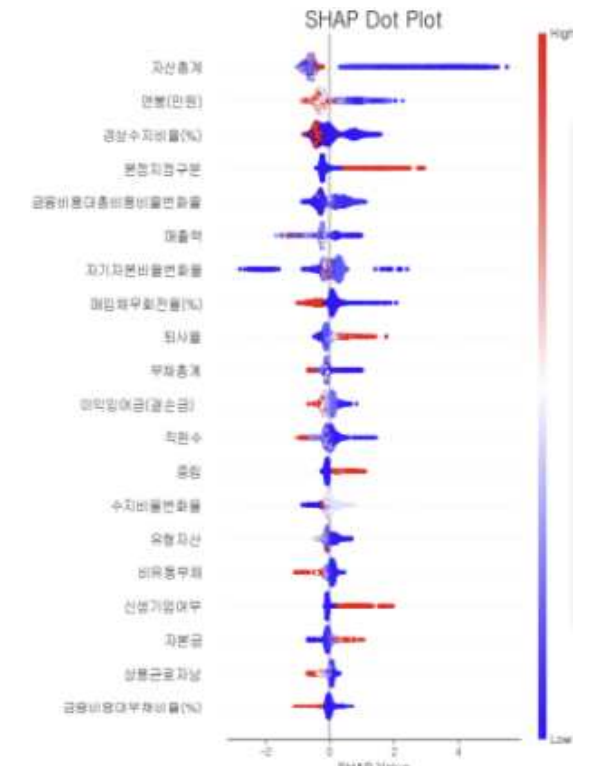
- 숫자형 데이터 처리
 - 결측치가 있는 기업을 단순 제거 시 너무 많은 데이터가 소실되는 문제 발생
→ 각 피처의 특성을 고려하여 결측치는 0으로 대체
- **MICE를 사용**하여 조건부 분포를 활용해 결측치를 예측하고, 결측치의 불확실성과 분산을 반영하여 최종 데이터셋 구성하는 방식으로 다중 대체
- 기타 변수에 대해 평균 및 추가 범주를 생성하여 처리
- 서비스에서 시각화 될 정보들은 Lambda 모듈을 통해 RDS에 저장

[모델링 및 예측 - Batch]

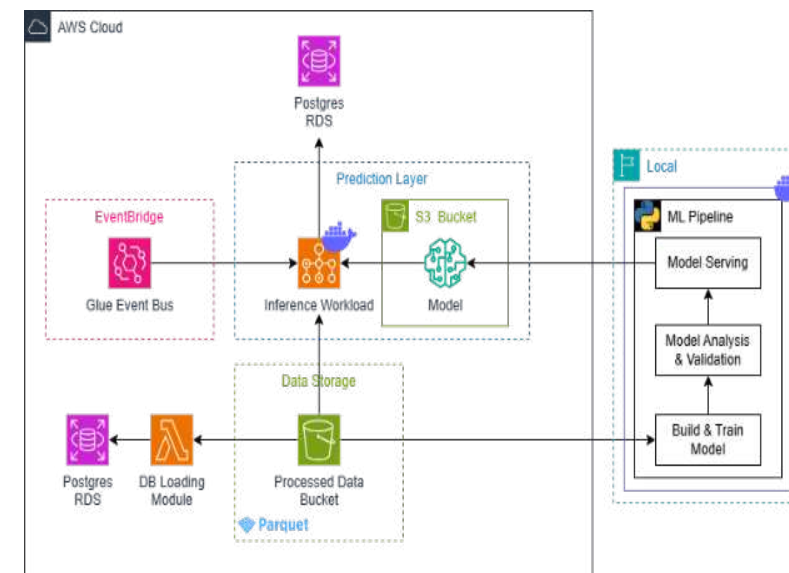
- **Feature Importances와 SHAP를 적용**해 어떤 어떤 변수가 성장율에 주요한 영향을 미치는지 분석한 후 예측 모델에 해당 값을 반영
- 개선된 모델에서 각 비재무적 요인이 모델에 미치는 영향을 해석하고 중요도를 파악



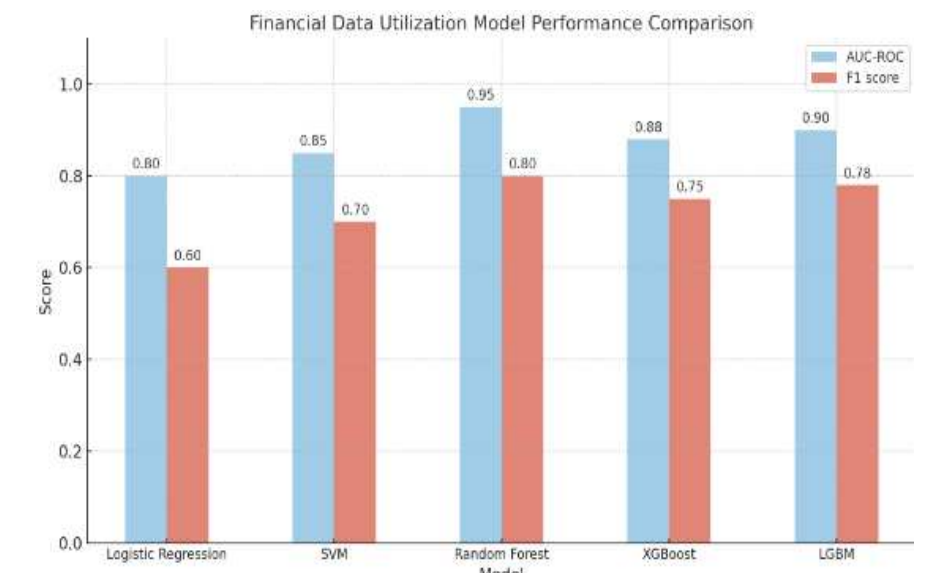
<연봉, 직원 수 등 데이터 처리>



<SHAP 중요도 해석 지표>

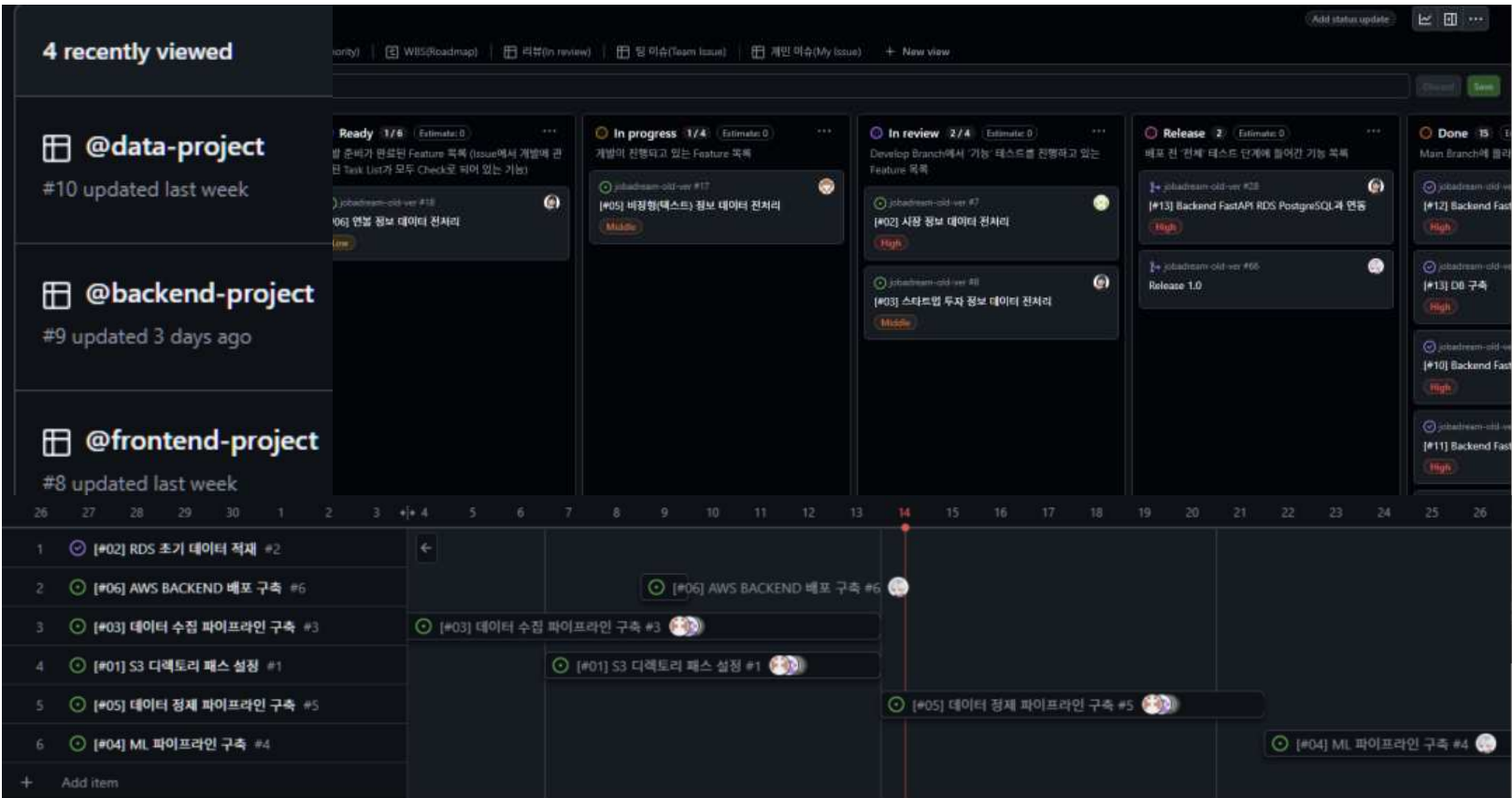


<모델링 및 예측 레이어>



<최종 사전 모델 성능 평가>

프로젝트 관리



칸반 보드, 작업 우선순위, WBS 관리, ISSUE / CODE REVIEW

각 팀마다 프로젝트를 분할시켜 해당 PROJECT LEADER(이하 PL)가 프로젝트 관리 및 유지보수를 할 수 있도록 프로젝트 구조 설계 및 관리
팀 단위 주간 스프린트 보고 및 프로젝트 단위 일간 스크럼 보고을 나눠 활발한 소통 진행

#업무_할당 #우선순위_배정 #일정_관리 #CODE CONVENTION

5.7. Database 설계

Database는 PostgreSQL를 사용한다.

5.7.1. ERD (Entity Relationship Diagram)

2. 프로젝트 작업 명세

2.1. 데이터 전처리

업 무	업무 범위
RAW 데이터 수집 (Lambda to S3)	- 수집할 데이터 소스를 식별 및 선정 - API 연동 및 크롤링/스크래핑을 이용하여 데이터 수집 - CSV포맷으로 수집 데이터 변환 - 수집 데이터를 AWS S3 버킷에 업로드
데이터 정제 (Ghle)	- 수집된 원시 데이터의 품질 검토 - 결측치 처리 - 데이터 정규화 - 중복 제거 - 변환 및 매핑
정제 데이터 보관 (to DB)	- 사전 작업 - 데이터 베이스 설계 - ERD 설계 - 테이블 설계 - 인덱스 및 쿼리 최적화 - 정제 데이터 보관

2.2. 검색 기능 개발

업 무	업무 범위
기업 정보 텍스트 검색	- 검색란을 통해 해당 기업명을 입력 시 해당 기업명에 유사한 기업 리스트 조회 - 키워드 전처리(토큰화, 정규화, 동의어 처리) - 검색 쿼리 처리 - 성능 최적화
카테고리별 상세 검색	- 각 테이블의 원형 데이터를 통해 상세 검색 기능 제공 - 카테고리 필드 및 구조 설계 - 검색 쿼리 구성 - 결과 필터링 및 정렬 - 성능 최적화

5.5. 요구사항

식별자	DR-01
명칭	재무 정보 데이터 전처리
책임자	FB
요구사항 상세설명	- DART(전자공시시스템)에서 기업 공시(재무제표) 정보 및 결산(연/분기)기준 재무비율에 대한 데이터 전처리 (최소 3년치) - 재무 정보 데이터 수집 - 수집데이터를 바탕으로 데이터 가공
수집데이터 목록	목록 - 자산 - 유동자산 - 비유동자산 - 자산총계 - 부채 - 유동부채 - 비유동부채 - 부채총계 - 자본 - 자본금 - 주식발행초과금 - 이익잉여금 - 기타자본항목 - 자본총계 - 자본과부채총계 - 손익계산서 - 매출액 - 매출원가 - 매출손익 - 판매비와관리비 - 영업이익 - 기타이익 - 기타비용 - 금융수익 - 금융비용 - 법인세비용차감전순이익(순실) - 법인세비용(수익) - 계속영이익(순실) - 분기순이익(순실) - 추당이익 (기본주당이익, 희석주당이익)

프로젝트 기획

프로젝트 개요, 추진 체계, 추진 계획, 작업 명세, 시스템 구조,
API 설계, 요구사항, DATABASE 설계, 스토리보드 등을 문서화
전반적인 프로젝트 기획은 본인이 담당하였으며 각 팀원들에게 기타 설계 업무를 배정함

#ISSUE #PULL REQUEST #ROLES #RULES

성과 및 피드백

정량적 성과

- 2024-10-01 기준 1차 AGILE 프로젝트 집계지표
 - 누적 사용자 1053명 집계 (오픈톡방, 각 학교 커뮤니티, 교육지점 등에 마케팅)
 - 각 기업에 대한 조회수 데이터 714건을 수집하여 사용자의 관심 기업에 대해 집계
- 2707개의 상장기업 및 10074개의 스타트업 데이터 수집 (재무제표, 비재무제표, 뉴스, 거시경제지표 등)
- 회고(RETROSPECTIVE) 과정을 통해 업무 프로세스 개선하여, 1차 AGILE 대비 WBS 진행도 20% 향상

정성적 성과

- 기존 프로젝트 대비 다수 인원의 팀 조직 관리 경험
- AWS 시스템/데이터 아키텍처 설계 및 구축 경험
- GIT PROJECT를 이용한 고유 프로젝트 관리 경험
- SCRUM 미팅 및 GIT 이슈 관리를 통해 팀 간의 정보 공유와 협업 완화, 의사소통 문제로 인한 업무 지연이 기존 프로젝트 대비 크게 감소
- 기획, 설계 문서화

피드백

- [AWS]
- LAMBDA CI/CD PIPELINE 구축 필요
 - ECS 배포 방식을 다음과 같은 이유로 블루/그린 배포 방식으로 전환 고려
 - A/B 테스트로 활용 가능
 - 문제 상황 빠르게 감지
- [데이터]
- 수집한 서비스 데이터를 이용하여 3차 AGILE에서 AARRR 분석 계획
 - 추가 수집 데이터를 활용하여 모델 성능 개선 필요

BTDS

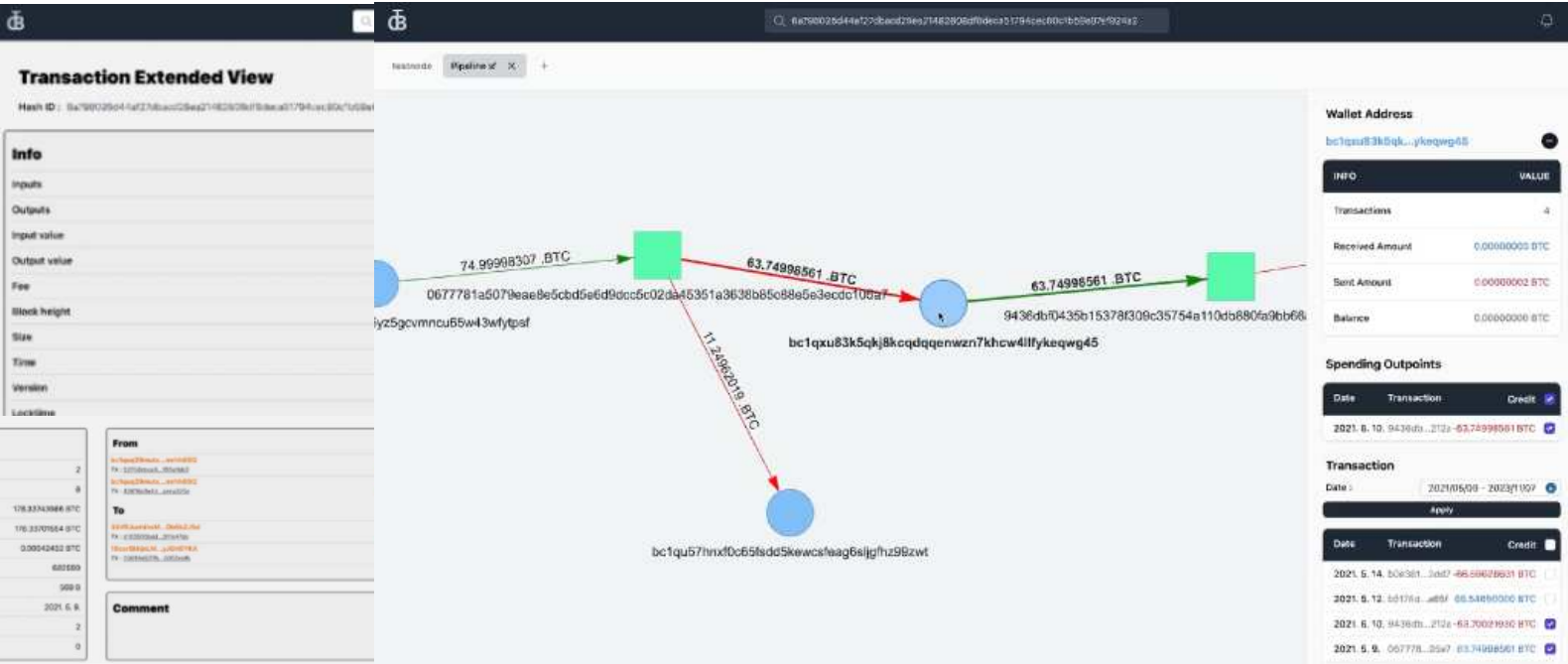
비트코인 사이버 범죄 추적 수사 지원 솔루션

개발 인원 : 4명

기여도 : 25% (PE/Process API 개발, 시각화 모듈 개발)

개발 기간 : 2023.05.15 ~ 2023.12.22 (약 7개월)

비트코인 사이버 범죄 추적 수사 지원 솔루션



[GITHUB](#)

[기술 스택] I Used

언어 : JavaScript TypeScript C++
프레임워크 : Next.js
데이터베이스 : MongoDB Neo4J
Devops : AWS Git/Github Notion.so JIRA Zeplin Bitcoin-core NGINX Atlas

[개요]

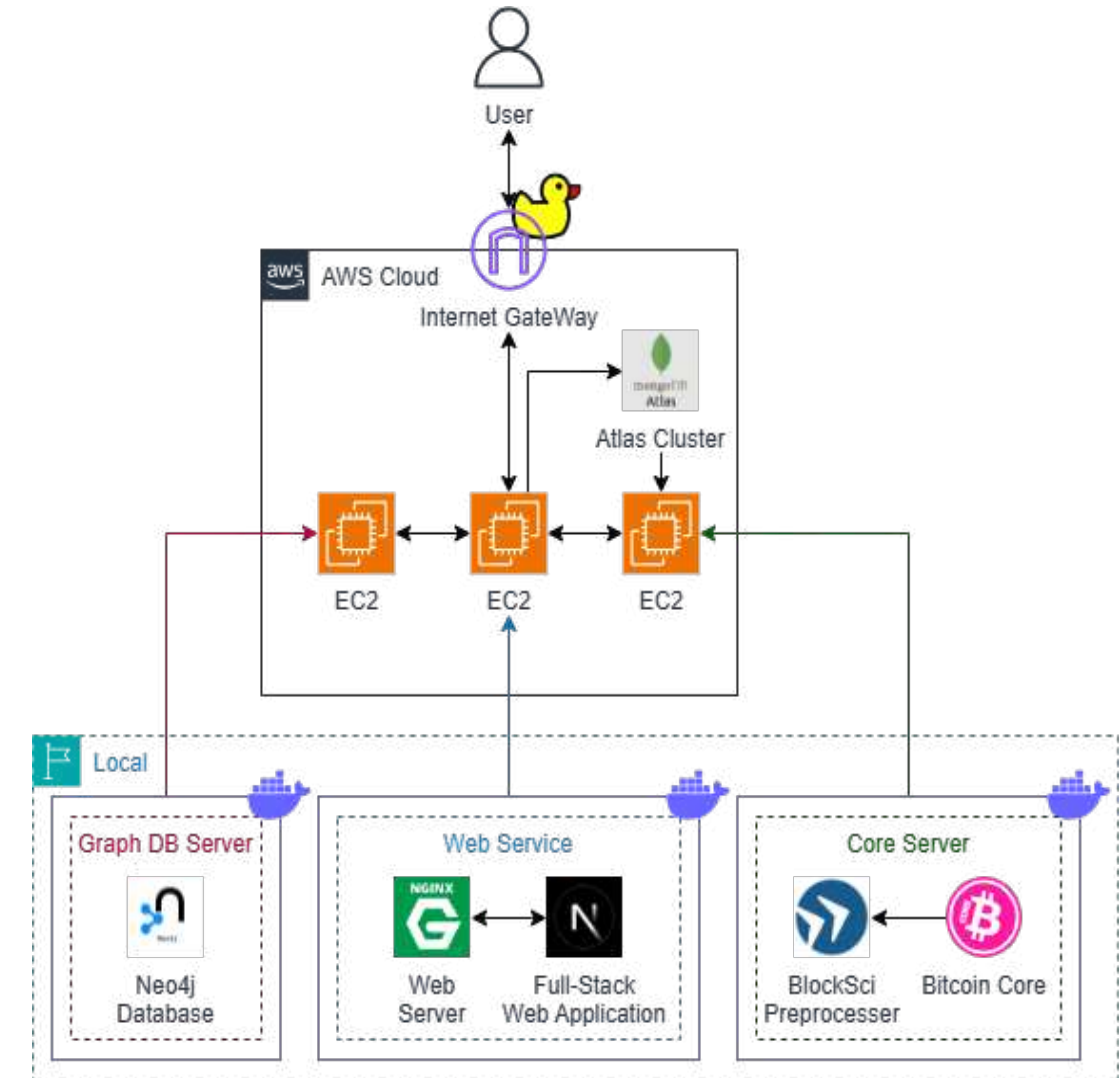
- 고용노동부가 주관하는 '미래내일 일경험 사업'으로 진행
- 9억여 건에 달하는 트랜잭션 데이터 수집 및 전처리
- 비트코인 거래가 진행된 사이버 범죄 추적 시 그래프 형태로 시각화
- 단순화된 데이터를 바탕으로 범죄 수사의 효율을 높이고 자동 추적 기능을 통하여 거래의 흐름을 신속히 분석

[담당 역할]

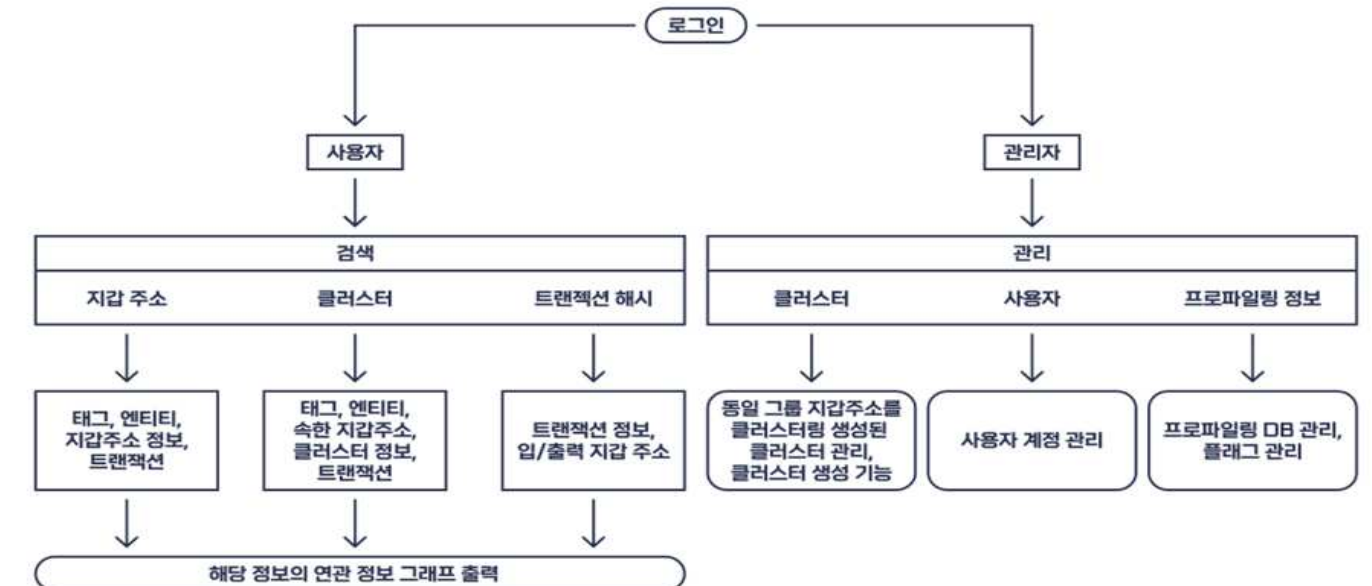
- C++을 사용한 Process API 개발
 - Bitcoin Core 풀 노드를 운영하여 트랜잭션 데이터를 수집
 - BlockSci를 사용한 비트코인 트랜잭션 데이터 분석 및 전처리
 - 휴리스틱 기법을 활용한 점수 기반 결정 모듈 개발
- Next.js와 vis.js를 사용한 네트워크 시각화(Node Graph) 모듈 개발

아키텍처 & 흐름도

1. 사용자가 웹 애플리케이션을 통해 비트코인 트랜잭션을 조회하거나 특정 주소 정보를 요청
2. 요청된 데이터는 웹 서버에서 API 호출을 통해 비트코인 블록체인에 저장된 데이터를 조회하기 위한 BlockSci를 사용
3. 트랜잭션 해시를 입력받아 블록체인 상에서 해당 트랜잭션을 조회하고, 입력값과 출력값, 수수료, 수신 주소 등을 MongoDB로부터 추가로 조회하여 클라이언트에게 반환
4. MongoDB에서 비트코인 주소나 트랜잭션에 대한 메타데이터 및 프로필 데이터를 저장하고 제공
5. 주어진 비트코인 주소에 대한 트랜잭션 내역을 조회하고, 관련 정보를 MongoDB에서 가져와 추가 분석
6. neo4j에서 그래프 형태로 노드 간의 관계를 분석하여 비트코인 주소 간의 연결 및 거래 네트워크를 시각적으로 표현
7. vis.js 라이브러리를 사용해 프론트엔드에서 이러한 네트워크 시각화를 처리하고, 이를 웹 인터페이스로 사용자에게 제공



시스템 흐름도



실험

[문제 & 목표]

- 문제 : 블록체인 데이터 및 거래 흐름 구조가 복잡하여 가상화폐 범죄 추적이 힘들다.
- 목표 : 비트코인 거래 내역을 쉽게 추적할 수 있도록 거래 흐름을 시각화하여 제공

[가설]

- 거래 추적에 가장 큰 어려움을 주는 것은 잔돈지갑의 유무 때문이 크다. 따라서 잔돈 지갑을 식별해낸다면 거래 흐름 추적 정확도가 상승될 것이다.
- 주소 재사용, 동일한 주소 포맷, 가장 큰 입력 금액, 어림수 등의 여러 휴리스틱 알고리즘을 사용한다면 잔돈지갑 주소 식별률이 높아질 것이다.

[실험 및 검증]

- 트랜잭션 대상을 무작위 선별하여 **휴리스틱을 적용**시켜보고 적중률이 높은 순서대로 랭크를 매겨 그룹을 3개로 나눔.
- 검증 결과 동일한 주소 포맷, 주소 재사용 휴리스틱의 적중률이 가장 높음.
- 검증 내용을 바탕으로 각 휴리스틱 그룹마다 점수 가중치를 다르게 주어 최종적으로 **점수 기반 결정**을 내려 정확도 향상
- 클러스터링 단일 사용 때보다 점수 기반 결정 로직 추가 사용 때의 적중률이 약 **12%P** 더 높음.



4. 관련 기술 동향 분석

4.1. 개요

비트코인 거래의 모든 거래 내역은 공개되어 있고 누구나 확인 가능. 거래 내역을 직접 보고 거래 흐름 파악하기에는 상당한 어려움이 있다. 이를 해결하기 위해 다양한 클러스터링을 이용한 거래 흐름을 분석하는 연구가 진행되고 있다. 이 연구에서는 클러스터링을 이용한 거래 흐름 분석을 위한 방법론을 제안한다.

4.2. Clustering

클러스터링은 동일한 그룹이라고 판단 되는 여러 주소를 묶는 과정이다. 이를 통해 거래 흐름을 단순화 시켜 추적을 용이하게 한다. 클러스터링을 하기 위한 방법론으로 Change address heuristic, Multi-Off-Chain 데이터 등을 이용한 클러스터링 방법이 있다.

Change address heuristic은 일회성 주소 사용자를 찾아내고 판단하는 방법이다.

Multi-Off-Chain heuristic은 모든 입력 주소의 소유자를 찾아내고 판단하는 방법이다.

4.3. 유사한 클러스터 분석

4.3.1. Crystal Explorer

Crystal Explorer는 비트코인 노드의 Crystal Blockchain에서 제공하는 클러스터링 도구이다. 이 도구를 통해 클러스터링을 쉽게 할 수 있다.

```
blocksci::heuristics::LocktimeChange locktimeChange;
blocksci::heuristics::AddressReuseChange addressReuseChange;
blocksci::heuristics::ClientChangeAddressBehaviorChange clientChangeBehaviorChange;
blocksci::heuristics::LegacyChange legacyChange;
blocksci::heuristics::FixedFee fixedFeeChange;
blocksci::heuristics::Spent spentChange;

for(const auto &output: tx.outputs()){
    outputScores[output] = 0;
}

for (const auto &item : addressReuseChange(tx)) {
    outputScores[item] += ADDRESS_REUSE_SCORE;
}

for (const auto &item : peelingChainChange(tx)) {
    outputScores[item] += PEELING_CHAIN_SCORE;
}

for (const auto &item : powerOfTenChange(tx)) {
```

<사전 연구 자료 조사>

<휴리스틱 흐름 추적 코드 일부>

<휴리스틱 흐름 추적 - 주소 재사용>

From 1 14ZgJpUj1eNQs9451X1WHsDbEeydrg9L 0.00453703 BTC + \$110.61

To 1 bc1qp54uk2paqt24ewlvxyt1z99lw6z027lqgs7q 0.00443703 BTC + \$108.17

Change 2 14ZgJpUj1eNQs9451X1WHsDbEeydrg9L 0.00009548 BTC + \$2.33

From 1 14ZgJpUj1eNQs9451X1WHsDbEeydrg9L 0.00453703 BTC + \$110.61

To 1 bc1qp54uk2paqt24ewlvxyt1z99lw6z027lqgs7q 0.00443703 BTC + \$108.17

Change 2 14ZgJpUj1eNQs9451X1WHsDbEeydrg9L 0.00009548 BTC + \$2.33

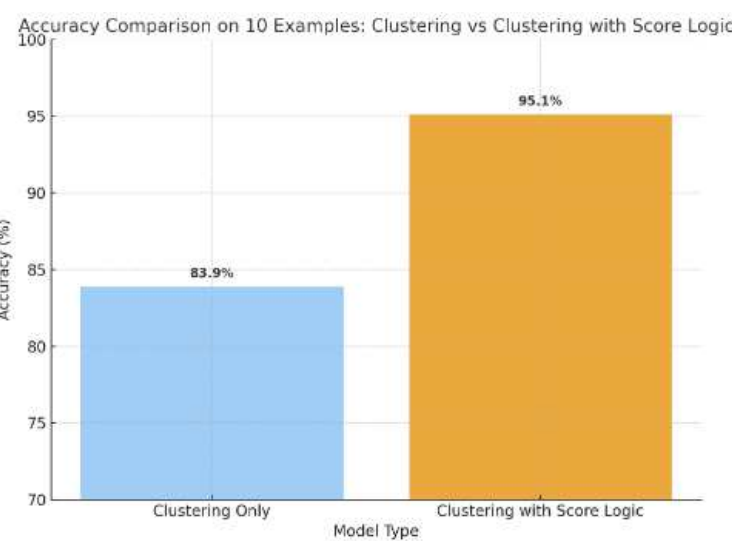
<휴리스틱 흐름 추적 - 어림수>

From 1 1KNyLmKojVhRSokMuq7zRc4r7Kqy2wvSKX 0.00423000 BTC + \$103.71

To 1 bc1qy92qyxf6kk7337shsev4d7lfr9l3y8t4k3wvm 0.00418000 BTC + \$102.48

Change 2 1Huk1ABCv2SPZcWHagYq8ZyFhWEwp5gtZM 0.00000706 BTC + \$0.17

<휴리스틱 흐름 추적 - 동일한 주소 포맷>



<10개의 케이스에 대한 노드 적중률 비교>

성과 및 피드백

정량적 성과

- 비동기 작업 병렬 처리, 부분 조회, 주 데이터 캐싱 처리로 데이터 처리 시간 1분 → 1초 감소
- Bitcoin Core RPC에서 BlockSci로 프로세서를 변경
 - 1. 데이터 접근 속도 50~100배 향상
 - 2. 주소 기반 트랜잭션 검색 속도 1~2분에서 1~2초로 단축
- AWS Neptune에서 Neo4j로 Graph DB를 변경하며 AWS 부가 비용을 감축시켜 약 20%~50%의 비용 절감
- 2023 상반기 프로젝트 경진대회 최우수상

정성적 성과

- 기획서, 설계서, 요구사항 명세서, 보고서 등과 같은 개발 문서화 작업 경험
- Jira를 통한 작업 관리 진행 경험
- 팀 내부 코드 스타일 및 네이밍 규칙을 지정해보며 협업 능력 향상
- vis.js를 사용한 그래프 형태 가시화
- 다양한 AWS 서비스(EC2, Neptune 등)를 사용/시도한 프로젝트 경험
- 대용량 트래픽을 FE/BE에서 처리해보는 경험

피드백

- 코어 서버가 무거워 성능 모니터링이 필요하다고 판단됨
→ AWS CLOUDWATCH와 같은 도구를 사용해 EC2 인스턴스와 각종 데이터베이스의 성능을 모니터링하고, 리소스 사용량에 대한 분석을 통해 필요한 부분에 최적화 진행 필요.
- 사용자 교육 문서 및 가이드 필요
→ 수사 기관 사용자가 시스템의 기능을 원활하게 사용할 수 있도록 사용자 가이드 및 교육 자료를 문서화하고 제공하여 도구 적응에 필요한 시간을 단축시킬 수 있다고 보임.

vAlscan BOX

AI기반 멀웨어 탐지 기능 통합 클라우드 스토리지 서비스

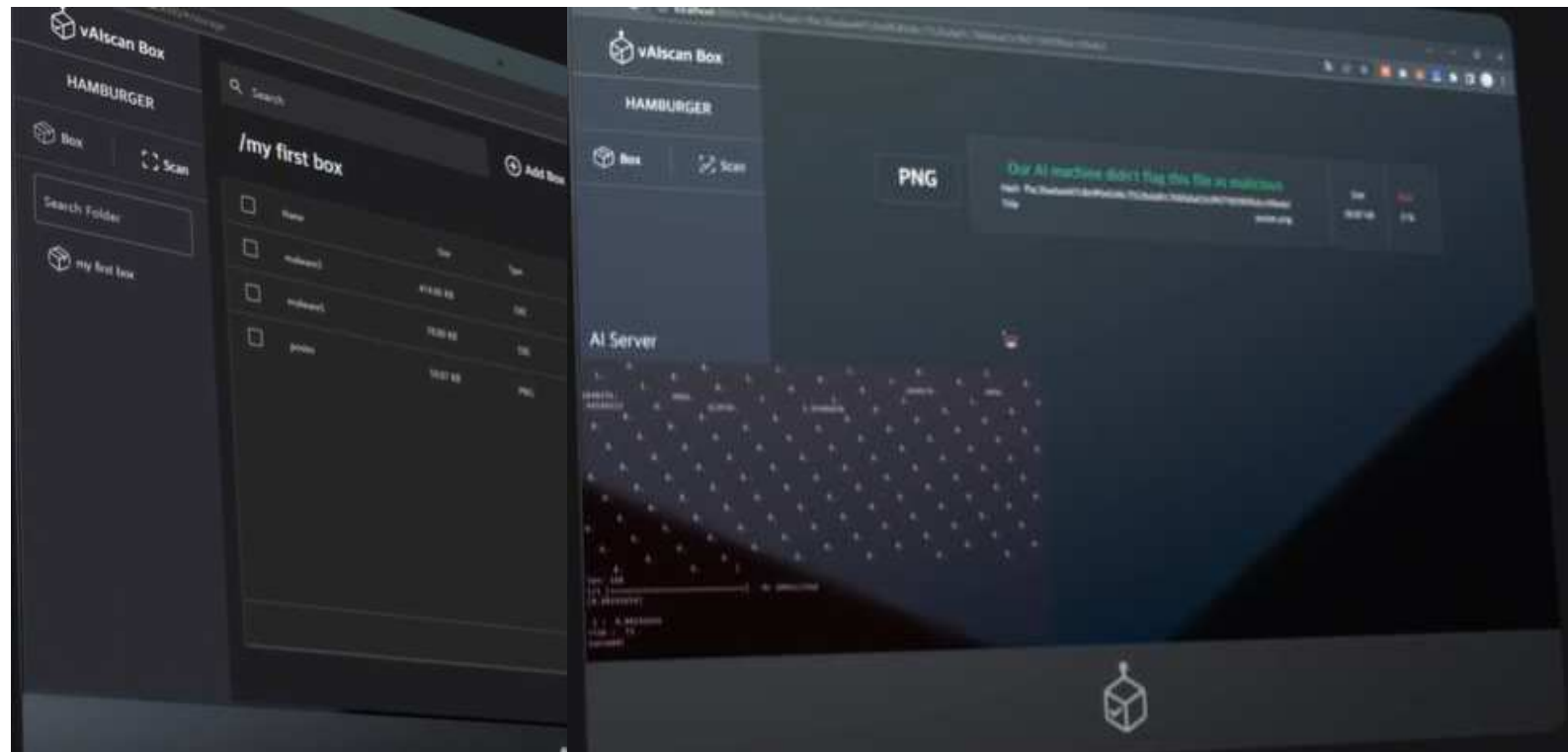
개발 인원 : 3명

기여도 : 40% (PE/데이터 수집, 모델 개발, 시각화 모듈 개발)

개발 기간 : 2022.12.01 ~ 2023.02.28 (3개월)

vAlscan Box

AI기반 멀웨어 탐지 기능 통합 클라우드 스토리지 서비스



[GITHUB](#)

[기술 스택] I Used Team Used

언어 : Python JavaScript TypeScript

프레임워크 : Capstone Vue.js FastAPI Nest.js

데이터베이스 : MongoDB MariaDB

Devops : AWS Git/Github Docker NGINX Zeplin Atlas

[개요]

- 비회원일 시에는 파일 검사 기능만 회원일 시에는 추가적으로 AWS S3 클라우드 스토리지 저장 옵션 제공
- 기존 시그니처 기반 탐지 방식 뿐 아니라 신규 유형의 멀웨어 탐지도 가능한 서비스로 파일의 해시 값을 이용하여 이미 검사된 파일은 DB에서 즉시 결과를 제공
- 신규 파일은 AI 서버에서 위험도를 분석한 후 결과 전달

[담당 역할]

- Python을 사용한 멀웨어 탐지 모델 개발
 - Capstone을 이용한 어셈블리 코드 분석 및 특징 분류
 - Keras를 사용한 CNN/DNN 모델 개발 및 비교
- Vue.js를 사용한 멀웨어 탐지 결과 시각화

아키텍처

04

1. Client가 파일 전송

2. 및 결과 조회

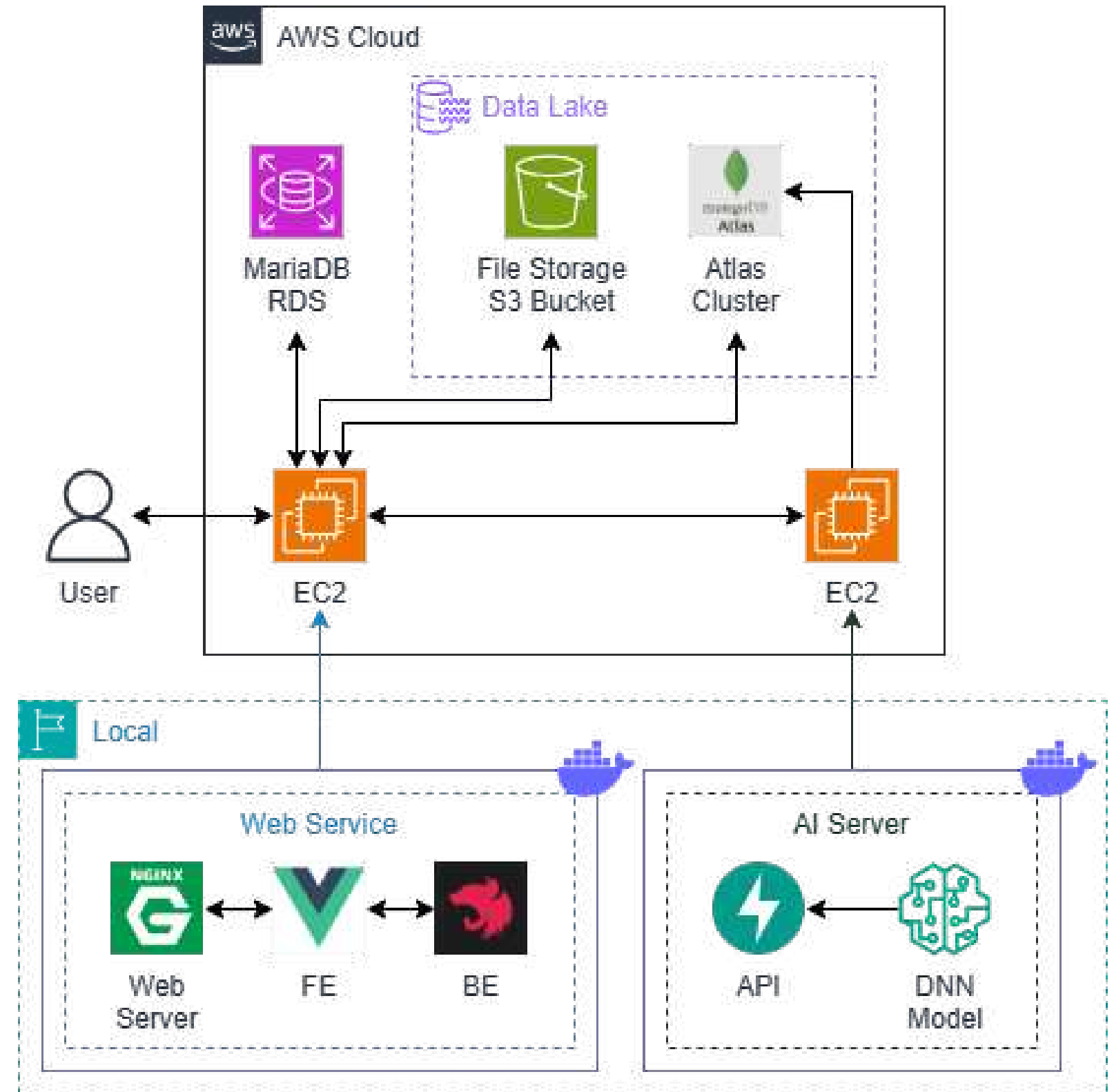
○ [이전에 검사한 결과가 있을 경우]

- i. MariaDB 서버에 이전 검사 기록이 있는지 조회
- ii. 검사 기록이 존재할 경우 MongoDB에 있는 이전 검사 기록 조회

○ [이전에 검사한 결과가 없을 경우]

- i. 두 DB에 파일의 hash 정보를 각각 저장
- ii. 받은 파일을 AI 서버에 전달
- iii. 해당 파일의 **PE Header & Code Section & Real File Type 분석**
- iv. 예측 결과를 업데이트하여 MongoDB에 저장
- v. DB에서 결과 조회

3. 멀웨어 예측 결과 페이지 출력



실험

[문제 & 목표]

- 문제 : 기존의 시그니처 기반의 멀웨어 탐지 방식은 새로운 유형의 멀웨어를 탐지하는데 한계가 있다.
- 목표 : AI 모델을 활용하여 새로운 멀웨어 패턴도 탐지할 수 있게 한다.

[가설]

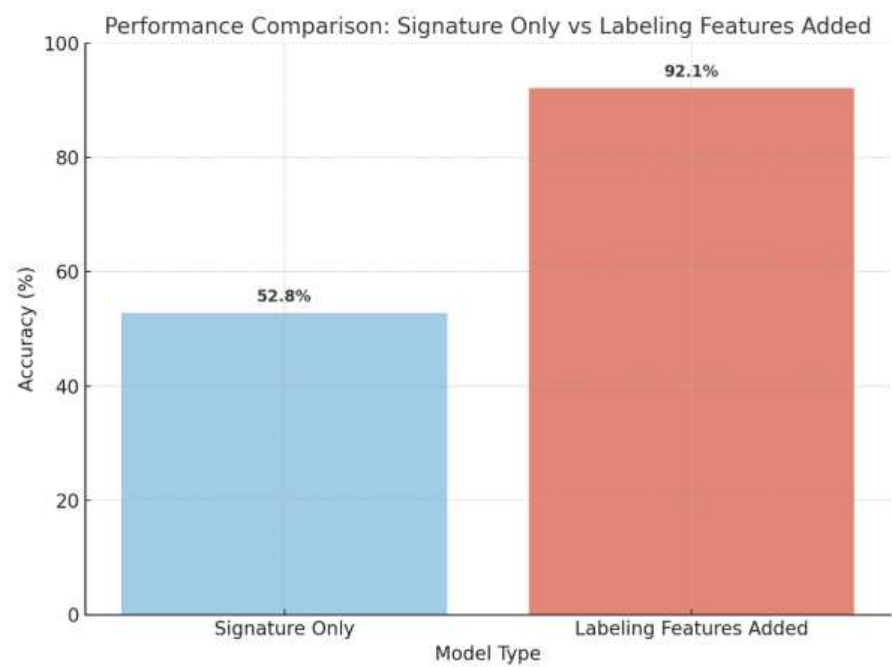
- PE Header 정보와 Code Section의 특징을 활용하면, 멀웨어 탐지의 정확도가 향상될 수 있다.

[실험 및 검증]

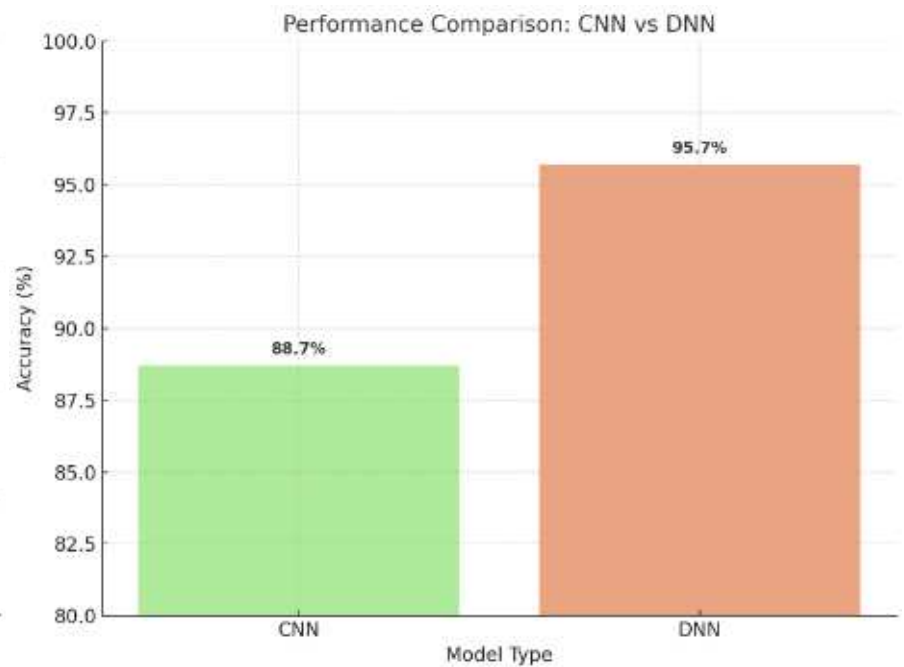
- 전처리 : 각 파일의 PE Header와 Code Section을 분석하여 특징을 벡터화
 - Capstone을 사용한 N-gram 방식의 어셈블리 코드 분석 및 특징 분류
 - 수집한 특징을 사용하여 멀웨어 라벨링
- 시그니처 패턴만 학습시켰을 때보다 특징 라벨링을 추가로 학습 시켰을 때 40%P 더 높은 정확도를 보임.
- 전처리 데이터를 활용하여 CNN/DNN 모델을 학습시킨 후, 성능 검증 및 비교
 - DNN이 CNN보다 7%P 더 높은 정확도를 보임.

filename	SHA256	e_cblp	e_cp	e_cparhdr	e_maxalloc	e_sp	filename	SHA256	mov	mov	mc	int3	int3	int3	push	push	pc	add	add	add	mov	mov	mi
55#a89cff731a89cff7301c		144	3	4	65535	184	55#a89cff731a89cff7301c		0	0													0
51#3704a8e3704a8eab23		144	3		65535	184	51#3704a8e3704a8eab23		0														0
52#3a17946i3a17946ac					65535	184	52#3a17946i3a17946ac																0
58#03ca2aaa03ca2aa					5535	184	58#03ca2aaa03ca2aa																20
45#87c0d32f87c0d3					35	184	45#87c0d32f87c0d3																18
57#662fa69a6						184	57#662fa69a6																141
54#ac7be3e0ac						184	54#ac7be3e0ac																0
56#ba07e07i ba07e					35	184	56#ba07e07i ba07e																0
47#96b9cd3f96b9cd304					5535	184	47#96b9cd3f96b9cd304																22
53#4db3b674db3b67780i		144	3	4	65535	184	53#4db3b674db3b67780i																29
52#151671c151671c3f1c		144	3	4	65535	184	52#151671c151671c3f1c																0
							18453#1a0a24611a0a24																6

<정적 데이터 분석 및 특징 분리>



<시그니처 학습 및 라벨링 추가 학습 비교>



<DNN과 CNN의 정확도 비교>

성과 및 피드백

정량적 성과

- 1000여 개의 프로그램 수집
- 10004개의 4-gram 어셈블리 코드 데이터 추출
- 9555개의 PE 데이터 추출
- PE Header 정보 중 68개의 특징 추출
- Code Section의 빈출있는 100개의 특징 추출
- 모델 멀웨어 탐지 정확도 95.78%
- 2022 하반기 프로젝트 경진대회 최우수상

정성적 성과

- PE HEADER 정보 분석 경험
- N-GRAM 방식의 어셈블리 코드 분석 경험
- 수집한 특징 기반 멀웨어 라벨링 경험
- 시그니처 방식의 단점을 보완하여 미래에 발생할 수 있는 새로운 위협에도 대응할 수 있는 보안 솔루션 제시

피드백

- 알림 기능을 도입하여 주요 노드에 대한 기능 추가 고려

PARK, JOONG-HEON PORTFOLIO

검토해 주셔서 감사합니다.

 **HANK YOU**
J O O N G - H E O N

CONTACT

MAIL : ORATORIA989@GMAIL.COM

더 많은 프로젝트 내역을 보고 싶으시다면 하단 링크로 접속해주세요

[PROJECTS](#)